



Sistemas Operativos

Clase 16 - Semáforos

Danny Múnera

Ingeniería de Sistemas

Universidad de Antioquia

Agenda

- Semáforos
- Problemas clásicos de sincronización
- Implementación

Semáforos: Definición

- Objeto con un valor entero
 - Se manipula a través de dos rutinas: `sem_wait()` y `sem_post()`
 - Inicialización:

```
1  #include <semaphore.h>
2  sem_t s;
3  sem_init(&s, 0, 1); // initialize s to the value 1
```

- Declara el semáforo **s** y lo inicializa en 1
- El segundo argumento indica que el semáforo es compartido entre los hilos de un mismo proceso.

Interacción con el semáforo

- `sem_wait()`

```
1  int sem_wait(sem_t *s) {  
2      decrement the value of semaphore s by one  
3      wait if value of semaphore s is negative  
4  }
```

- Si el valor del semáforo es **mayor o igual a uno**, la función **retorna inmediatamente**.
- En caso contrario, el **hilo llamador se suspende** a la espera de una señal de post.
- Un semáforo con un **valor negativo** indica el **número de hilos en espera**.

Interacción con el semáforo

- `sem_post()`

```
1  int sem_post(sem_t *s) {  
2      increment the value of semaphore s by one  
3      if there are one or more threads waiting, wake one  
4  }
```

- **Incrementa el valor** del semáforo
- Si hay hilos a la espera en el semáforo, **despierta alguno de ellos.**

Semáforos binarios: Locks

```
1  sem_t m;  
2  sem_init(&m, 0, X); // initialize semaphore to X; what should X be?  
3  
4  sem_wait(&m);  
5  //critical section here  
6  sem_post(&m);
```

¿Cuál debe ser el valor de **X**?

Ejemplo: 1 hilo, 1 semáforo

Value of Semaphore	Thread 0	Thread 1
1		
1	call sema_wait()	
0	sem_wait() returns	
0	(crit sect)	
0	call sem_post()	
1	sem_post() returns	

Ejemplo: 2 hilos, 1 semáforo

Value	Thread 0	State	Thread 1	State
1		Running		Ready
1	call sem_wait()	Running		Ready
0	sem_wait() retruns	Running		Ready
0	(crit set: begin)	Running		Ready
0	Interrupt; Switch → T1	Ready		Running
0		Ready	call sem_wait()	Running
-1		Ready	decrement sem	Running
-1		Ready	(sem < 0) → sleep	sleeping
-1		Running	Switch → T0	sleeping
-1	(crit sect: end)	Running		sleeping
-1	call sem_post()	Running		sleeping
0	increment sem	Running		sleeping
0	wait(T1)	Running		Ready
0	sem_post() returns	Running		Ready
0	Interrupt; Switch → T1	Ready		Running
0		Ready	sem_wait() retruns	Running
0		Ready	(crit sect)	Running
0		Ready	call sem_post()	Running
1		Ready	sem_post() returns	Running

Semáforos como variables de condición

```
1  sem_t s;
2
3  void *
4  child(void *arg) {
5      printf("child\n");
6      sem_post(&s); // signal here: child is done
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     sem_init(&s, 0, X); // what should X be?
13     printf("parent: begin\n");
14     pthread_t c;
15     pthread_create(&c, NULL, child, NULL);
16     sem_wait(&s); // wait here for child
17     printf("parent: end\n");
18     return 0;
19 }
```

¿Valor de **X**?

Hilo padre a la espera de un hijo

Padre llama `sem_wait()` antes que hijo llame `sem_post()`

Value	Parent	State	Child	State
0	Create (Child)	Running	<i>(Child exists; is runnable)</i>	Ready
0	call <code>sem_wait()</code>	Running		Ready
-1	decrement sem	Running		Ready
-1	$(sem < 0) \rightarrow \text{sleep}$	sleeping		Ready
-1	<i>Switch $\rightarrow$ Child</i>	sleeping	child runs	Running
-1		sleeping	call <code>sem_post()</code>	Running
0		sleeping	increment sem	Running
0		Ready	wake (Parent)	Running
0		Ready	<code>sem_post()</code> returns	Running
0		Ready	<i>Interrupt; Switch $\rightarrow$ Parent</i>	Ready
0	<code>sem_wait()</code> retruns	Running		Ready

Hilo padre a la espera de un hijo

Padre llama `sem_wait()` después de hijo llame `sem_post()`

Value	Parent	State	Child	State
0	Create (Child)	Running	(Child exists; is runnable)	Ready
0	<i>Interrupt; switch→Child</i>	Ready	child runs	Running
0		Ready	call <code>sem_post()</code>	Running
1		Ready	increment sem	Running
1		Ready	wake (nobody)	Running
1		Ready	<code>sem_post()</code> returns	Running
1	parent runs	Running	<i>Interrupt; Switch→Parent</i>	Ready
1	call <code>sem_wait()</code>	Running		Ready
0	decrement sem	Running		Ready
0	(sem<0)→awake	Running		Ready
0	<code>sem_wait()</code> retruns	Running		Ready

Problemas de sincronización

Problema de Productor/Consumidor

```
1  int buffer[MAX];
2  int fill = 0;
3  int use = 0;
4
5  void put(int value) {
6      buffer[fill] = value;    // line f1
7      fill = (fill + 1) % MAX; // line f2
8  }
9
10 int get() {
11     int tmp = buffer[use];    // line g1
12     use = (use + 1) % MAX;    // line g2
13     return tmp;
14 }
```

Solución 1 con semáforos

```
1  sem_t empty;
2  sem_t full;
3
4  void *producer(void *arg) {
5      int i;
6      for (i = 0; i < loops; i++) {
7          sem_wait(&empty);           // line P1
8          put(i);                     // line P2
9          sem_post(&full);            // line P3
10     }
11 }
12
13 void *consumer(void *arg) {
14     int i, tmp = 0;
15     while (tmp != -1) {
16         sem_wait(&full);             // line C1
17         tmp = get();                 // line C2
18         sem_post(&empty);            // line C3
19         printf("%d\n", tmp);
20     }
21 }
22 ...
```

Problema de la Solución 1

```
21  int main(int argc, char *argv[]) {  
22      // ...  
23      sem_init(&empty, 0, MAX);           // MAX buffers are empty to begin with...  
24      sem_init(&full, 0, 0);              // ... and 0 are full  
25      // ...  
26  }
```

- $MAX > 1$, y existen múltiples productores
 - Race condition en función `put` (línea f1)
- Exclusión mutua!
 - Llenar el buffer e incrementar el contador es una sección crítica

Solución 2: Agregando Exclusión Mutua

```
1  sem_t empty;
2  sem_t full;
3  sem_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          sem_wait(&mutex);           // line p0 (NEW LINE)
9          sem_wait(&empty);           // line p1
10         put(i);                     // line p2
11         sem_post(&full);             // line p3
12         sem_post(&mutex);           // line p4 (NEW LINE)
13     }
14 }
15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++) {
19         sem_wait(&mutex);           // line c0 (NEW LINE)
20         sem_wait(&full);             // line c1
21         int tmp = get();             // line c2
22     }
23 }
```


Solución 2: Agregando Exclusión Mutua

```
22         sem_post(&empty);           // line c3
23         sem_post(&mutex);           // line c4 (NEW LINE)
24         printf("%d\n", tmp);
25     }
26 }
```

- Un hilo productor y un hilo consumidor
 - Consumidor adquiere el **mutex**
 - Consumidor llama `sem_wait()` en el semáforo **full**
 - **Consumidor está bloqueado** y libera la CPU
 - El consumidor aún posee el **mutex**
 - Productor llama `sem_wait()` en semáforo **mutex**
 - **Productor está bloqueado**

Solución 2: Agregando Exclusión Mutua

```
22         sem_post(&empty);           // line c3
23         sem_post(&mutex);           // line c4 (NEW LINE)
24         printf("%d\n", tmp);
25     }
26 }
```

- Un hilo productor y un hilo consumidor
 - Consumidor adquiere el **mutex**
 - Consumidor llama `sem_wait()` en el semáforo **full**
 - **Consumidor está bloqueado** y libera la CPU
 - El consumidor aún posee el **mutex**
 - Productor llama `sem_wait()` en semáforo **mutex**
 - **Productor está bloqueado**

Interbloqueo (Deadlock)

Solución 3: Final

```
1  sem_t empty;
2  sem_t full;
3  sem_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          sem_wait(&empty);           // line p1
9          sem_wait(&mutex);           // line p1.5 (MOVED MUTEX HERE...)
10         put(i);                     // line p2
11         sem_post(&mutex);           // line p2.5 (... AND HERE)
12         sem_post(&full);            // line p3
13     }
14 }
15
16 void *consumer(void *arg) {
17     int i;
18     for (i = 0; i < loops; i++) {
19         sem_wait(&full);             // line c1
20         sem_wait(&mutex);           // line c1.5 (MOVED MUTEX HERE...)
21         int tmp = get();             // line c2
22         sem_post(&mutex);           // line c2.5 (... AND HERE)
23     }
24 }
```

Solución 3: Final

```
23         sem_post(&empty);           // line c3
24         printf("%d\n", tmp);
25     }
26 }
27
28 int main(int argc, char *argv[]) {
29     // ...
30     sem_init(&empty, 0, MAX); // MAX buffers are empty to begin with ...
31     sem_init(&full, 0, 0);    // ... and 0 are full
32     sem_init(&mutex, 0, 1);   // mutex=1 because it is a lock
33     // ...
34 }
```

Problema de Lectores- Escritores

- Acceso concurrente a datos, lecturas y escrituras
 - Escrituras:
 - Cambia el estado de los datos
 - Sección crítica
 - Lecturas
 - Realiza una lectura simple a los datos
 - Si no hay escrituras realizándose, se puede permitir múltiples lecturas concurrentes

Reader-Writer Lock

- Sólo un escritor puede adquirir el *lock*
- Cuando un lector adquiere el *lock*
 - Se permite que más lectores adquieran también el *lock*
 - Un escritor tiene que esperar a que todos los lectores finalicen

```
1  typedef struct _rwlock_t {
2      sem_t lock;           // binary semaphore (basic lock)
3      sem_t writelock;      // used to allow ONE writer or MANY readers
4      int readers;          // count of readers reading in critical section
5  } rwlock_t;
6
7  void rwlock_init(rwlock_t *rw) {
8      rw->readers = 0;
9      sem_init(&rw->lock, 0, 1);
10     sem_init(&rw->writelock, 0, 1);
11 }
12
13 void rwlock_acquire_readlock(rwlock_t *rw) {
14     sem_wait(&rw->lock);
15     ...
```

Reader-Writer Lock

```
15     rw->readers++;
16     if (rw->readers == 1)
17         sem_wait(&rw->writelock); // first reader acquires writelock
18     sem_post(&rw->lock);
19 }
20
21 void rwlock_release_readlock(rwlock_t *rw) {
22     sem_wait(&rw->lock);
23     rw->readers--;
24     if (rw->readers == 0)
25         sem_post(&rw->writelock); // last reader releases writelock
26     sem_post(&rw->lock);
27 }
28
29 void rwlock_acquire_writelock(rwlock_t *rw) {
30     sem_wait(&rw->writelock);
31 }
32
33 void rwlock_release_writelock(rwlock_t *rw) {
34     sem_post(&rw->writelock);
35 }
```

Reader-Writer Lock: problema de equidad

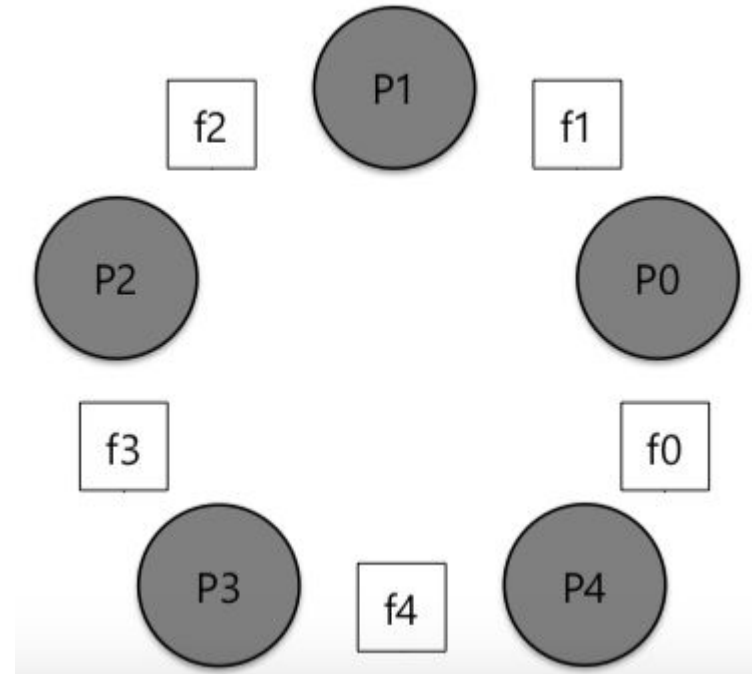
- Problema de equidad (*fairness*)
 - Muchos lectores deniegan el acceso al escritor (*starvation*)

Reader-Writer Lock: problema de equidad

- Problema de equidad (*fairness*)
 - Muchos lectores deniegan el acceso al escritor (*starvation*)
 - **Solución:** Evitar el acceso de nuevos lectores cuando haya un escritor en espera.

Cena de filósofos

- Se tienen 5 filósofos sentados en una mesa redonda
 - Entre cada filósofo hay un palillo chinos (5 en total)
 - Cada filósofo pasa su vida **pensando** (sin necesitar ningún palillo) o **comiendo**
 - Para comer cada filósofo necesita 2 palillos, el que tiene a su derecha y a su izquierda
 - Los filósofos compiten por estos palillos



Cena de filósofos

- Desafío

- No existan interbloqueos (*deadlocks*)
- No se quede ningún filósofo sin comer (*starvation*)
- Máxima **concurrency**

```
while (1) {  
    think();  
    getforks();  
    eat();  
    putforks();  
}
```

```
// helper functions  
int left(int p) { return p; }  
  
int right(int p) {  
    return (p + 1) % 5;  
}
```

- Filósofo p quiere adquirir palillo de la izquierda -> left (p) ;
- Filósofo p quiere adquirir palillo de la derecha -> right (p) ;

Cena de filósofos

- Se requiere un semáforo para cada palillo:
 - `sem_t forks[5];`

```
1  void getforks() {  
2      sem_wait(forks[left(p)]);  
3      sem_wait(forks[right(p)]);  
4  }  
5  
6  void putforks() {  
7      sem_post(forks[left(p)]);  
8      sem_post(forks[right(p)]);  
9  }
```

- Puede ocurrir un Interbloqueo (*Deadlock*)

Cena de filósofos

- Solución:
 - Cambiar el orden en el que los semáforos son adquiridos

```
1  void getforks() {  
2      if (p == 4) {  
3          sem_wait(forks[right(p)]);  
4          sem_wait(forks[left(p)]);  
5      } else {  
6          sem_wait(forks[left(p)]);  
7          sem_wait(forks[right(p)]);  
8      }  
9  }
```

Implementación de semáforos

Implementación de Zemáforos

```
1  typedef struct __Zem_t {
2      int value;
3      pthread_cond_t cond;
4      pthread_mutex_t lock;
5  } Zem_t;
6
7  // only one thread can call this
8  void Zem_init(Zem_t *s, int value) {
9      s->value = value;
10     Cond_init(&s->cond);
11     Mutex_init(&s->lock);
12 }
13
14 void Zem_wait(Zem_t *s) {
15     Mutex_lock(&s->lock);
16     while (s->value <= 0)
17         Cond_wait(&s->cond, &s->lock);
18     s->value--;
19     Mutex_unlock(&s->lock);
20 }
21 ...
```

Implementación de Zemáforos

```
22  void Zem_post(Zem_t *s) {  
23      Mutex_lock(&s->lock);  
24      s->value++;  
25      Cond_signal(&s->cond);  
26      Mutex_unlock(&s->lock);  
27  }
```

- El valor del semáforo en esta implementación no representa el número de hilos bloqueados en el semáforo (cuando el valor es negativo).
- Fácil implementación, similar a la implementación actual en linux.

Fin Clase 16



UNIVERSIDAD DE ANTIOQUIA

1 8 0 3

Referencias

- Operating Systems Three Easy Pieces
 - Capítulo 31

Esta presentación se ha realizado tomando como base las presentaciones proporcionadas por los autores del libro.